

Universidad Nacional Autónoma de México
Facultad de Ingeniería

Cómputo Móvil
Semestre 2026 - 2
Grupo: 03
Fecha de entrega: 29/05/2026



Trabajo Final.

Integrantes Hackatronicos:

- Guerrero Gutiérrez Héctor
- Miyasaki Sato Yuichi Vicente
- Sampayo Aguilar Cinthia Gloricel

Profesor: ING MARDUK PÉREZ DE LARA DOMÍNGUEZ

Tabla de contenido

Tabla de contenido	2
1. Requerimientos de la app	3
1.1 Reglas de negocio	3
1.2 Requerimientos funcionales	3
1.3 Requerimientos no funcionales	4
2. Viabilidad técnica de funcionalidades propuestas	4
2.1 Funcionalidades viables que sí implementamos	4
2.2 Funcionalidades inviables o postergadas	5
3. Alcance y MVP	5
3.1 Alcance incluido en el MVP	5
3.2 Alcance excluido del MVP	5
3.3 Criterios de aceptación del MVP	6
4. Wireframes de la app	6
5. Flujo de navegación	11
5.1 Flujos principales	11
5.2 Gestos especiales	11
6. Documentación por pantalla	12
6.1 W01. Dashboard	12
6.2 W02. Lista de gastos	12
6.3 W03. Nuevo gasto	12
6.4 W04. Escaneo de ticket	13
6.5 W06. Insights	13
6.6 W07. Resumen con IA	14
6.7 W08. Comparación mensual	14
6.8 W09 y W10. Ajustes y API key	14
7. Flujo de datos y cambios de estado	15
7.1 Capas de datos	15
7.2 Cambios de estado relevantes	15
8. Dispositivos, orientación y sensores	15
8.1 Dispositivos soportados	15
8.2 Orientación de pantalla	15
8.3 Sensores utilizados	16
9. Stack técnico y herramientas	16
9.1 Lenguajes de programación	16
9.2 Frontend	16
9.3 Backend y persistencia	16
9.4 Servicios externos	16
9.5 Herramientas de desarrollo	16
10. Arquitectura y metodologías	17
10.1 Arquitectura general	17
10.2 Patrones de diseño aplicados	17
10.3 Metodología de desarrollo	17

11. Administración del proyecto	17
11.1 Herramientas	17
11.2 Estructura del backlog	18
11.3 Roles dentro del Hack	18
12. Uso de IA en el desarrollo	18
12.1 Como asistente de programación	18
12.2 Como herramienta de diseño y diagramación	18
12.3 Como motor del producto final	18
13. Permisos y consideraciones para la publicación	18
13.1 Permisos del sistema	18
13.2 Privacy Nutrition Labels	18
13.3 Cumplimiento normativo	19
13.4 Requisitos previos a la publicación	19
14. Equipo de trabajo y roles	19
15. Estimaciones de tiempo y costos	20
15.1 Tiempo de desarrollo	20
15.2 Costos de desarrollo	20
15.3 Costos de operación e implementación	21
15.4 Costos de mantenimiento	21
16. Referencias	21
17. Anexos	21

1. Requerimientos de la app

En esta sección detallamos los requerimientos que definen el alcance funcional y operativo de GastoClaro AI. Los separamos en tres bloques: reglas de negocio, requerimientos funcionales y requerimientos no funcionales.

1.1 Reglas de negocio

Son las reglas que definen cómo opera el dominio del producto, independientes de la implementación técnica:

- **RN-01.** Un gasto siempre pertenece a exactamente una categoría. Si el usuario no asigna categoría, se asigna a "Otros" por defecto.
- **RN-02.** Toda categoría tiene una marca booleana `isAntExpense` que define si los gastos asociados se contabilizan como gastos hormiga para la proyección anual.
- **RN-03.** El presupuesto se define por mes y año y solo aplica para ese período. No se traslada al mes siguiente.
- **RN-04.** La proyección anual de gastos hormiga se calcula multiplicando el gasto hormiga del mes en curso por 12. Es una estimación, no un compromiso.
- **RN-05.** La alerta de "vas adelantado en tu presupuesto" se dispara cuando el porcentaje gastado supera al porcentaje de días transcurridos del mes en más de 10 puntos.
- **RN-06.** El resumen con IA solo se genera si el usuario tiene una API key válida configurada. Sin key, la opción se muestra deshabilitada con instrucción de cómo activarla.
- **RN-07.** Los datos enviados a OpenAI son únicamente agregados: totales por categoría, presupuesto y proyecciones. Nunca se envían descripciones individuales, comercios ni fechas exactas.
- **RN-08.** Las sugerencias automáticas del OCR son revisables: el usuario debe confirmar comercio, monto y categoría antes de guardar el gasto.
- **RN-09.** El borrado de gastos no es reversible una vez confirmado. El borrado masivo conserva las categorías.
- **RN-10.** La moneda base es el peso mexicano. No se permiten gastos con monto cero o negativo.

1.2 Requerimientos funcionales

Describen las capacidades concretas que la aplicación debe ofrecer al usuario:

ID	Requerimiento	Prioridad
RF-01	El usuario puede registrar un gasto manualmente con descripción, monto, fecha, categoría, método de pago y notas.	Alta
RF-02	El usuario puede escanear un ticket físico con la cámara y la app extrae automáticamente comercio, monto y categoría sugerida.	Alta
RF-03	El usuario puede consultar el listado completo de sus gastos, agrupados por día, con buscador y filtro por categoría.	Alta
RF-04	El usuario puede consultar el detalle de un gasto y eliminarlo.	Alta
RF-05	El usuario visualiza un dashboard con total del mes, avance del presupuesto, gastos hormiga del mes y proyección anual.	Alta
RF-06	El usuario puede definir un presupuesto mensual y la app le indica avance proporcional al día del mes.	Media

ID	Requerimiento	Prioridad
RF-07	El usuario consulta insights automáticos generados localmente: categoría dominante, alertas, promedio diario, patrones de café.	Media
RF-08	El usuario puede generar un resumen narrativo de su mes con GPT-4o mini de OpenAI, previa configuración de su API key.	Media
RF-09	El usuario visualiza la gráfica de barras de los 12 meses del año seleccionado y navega entre años con dos chevrons.	Media
RF-10	El usuario visualiza una gráfica circular con la distribución de gasto por categoría del mes actual.	Baja
RF-11	El usuario puede configurar y validar su API key de OpenAI, así como eliminarla.	Media
RF-12	El usuario puede borrar todos los gastos conservando categorías y presupuesto.	Baja

1.3 Requerimientos no funcionales

Definen restricciones de calidad, seguridad y rendimiento que debe cumplir la app:

- **RNF-01. Disponibilidad:** la app debe funcionar completamente sin conexión a internet, excepto la generación de resúmenes con IA.
- **RNF-02. Rendimiento:** el dashboard debe cargar en menos de 500 ms con hasta 10,000 gastos registrados; el OCR sobre una imagen debe completarse en menos de 3 segundos en iPhone 13 o superior.
- **RNF-03. Privacidad:** todos los datos personales del usuario se almacenan localmente con SwiftData; las API keys se guardan en Keychain con accesibilidad afterFirstUnlock.
- **RNF-04. Seguridad:** las llamadas a OpenAI usan HTTPS con TLS 1.3; la API key se envía solamente en el header Authorization y nunca se loggea.
- **RNF-05. Usabilidad:** todas las acciones críticas requieren máximo dos toques desde la pantalla principal. La interfaz cumple con los lineamientos de Apple Human Interface Guidelines.
- **RNF-06. Localización:** todos los textos están en español mexicano; los montos se muestran con formato de pesos mexicanos; las fechas usan formato local es-MX.
- **RNF-07. Accesibilidad:** soporte para VoiceOver, Dynamic Type y modos de alto contraste; ningún elemento interactivo menor a 44x44 puntos.
- **RNF-08. Compatibilidad:** iOS 17 o superior, iPhone 11 o posterior. Diseñada para orientación vertical principalmente, con soporte de orientación horizontal solo en pantallas de gráficas.

2. Viabilidad técnica de funcionalidades propuestas

Durante el proceso de definición evaluamos varias funcionalidades adicionales. A continuación documentamos cuáles consideramos viables y cuáles descartamos, con la justificación correspondiente.

2.1 Funcionalidades viables que sí implementamos

- **OCR de tickets con Vision:** viable. El framework Vision de Apple ofrece reconocimiento de texto multilingüe con precisión suficiente y latencia inferior a 2 segundos en iPhone 13. No requiere modelos personalizados ni infraestructura externa.
- **Clasificación automática por heurísticas:** viable. Un diccionario de palabras clave por categoría cubre la mayoría de comercios mexicanos comunes, con precisión observada superior al 70% en pruebas con tickets reales.

- **Resumen narrativo con GPT-4o mini:** viable. El modelo cuesta aproximadamente 0.15 dólares por millón de tokens de entrada; un resumen típico consume 800 tokens, lo que equivale a fracción de centavo por solicitud.
- **Persistencia con SwiftData:** viable. SwiftData en iOS 17 ofrece un API declarativo y rendimiento más que suficiente para el volumen esperado de datos personales del usuario.

2.2 Funcionalidades inviables o postergadas

- **Conexión directa con bancos mexicanos:** inviable en el corto plazo. Requiere convenios con cada institución bancaria y certificación bajo la Ley Fintech mexicana. Los agregadores como Belvo o Plaid cobran tarifas mensuales por usuario que rompen el modelo freemium. Postergado a fase 2 con marca blanca para un banco aliado.
- **Reconocimiento de voz para registrar gastos:** técnicamente viable con el framework Speech de Apple, pero descartado en la versión inicial porque requiere permisos adicionales del usuario, un flujo de UX dedicado y un parser que extraiga monto y categoría del lenguaje natural en español. El esfuerzo de validación supera el beneficio en el alcance del Hack.
- **Predicción de gasto futuro con modelo propio:** inviable por ahora. Construir un modelo de series temporales personalizado requiere historial de al menos 6 meses por usuario y entrenamiento por cohorte. Lo postergamos hasta tener masa crítica de datos y capacidad de entrenamiento.
- **Sincronización entre múltiples dispositivos:** viable técnicamente con CloudKit, pero implica un nivel de infraestructura y validación que no cabía en el tiempo del Hack. Lo dejamos para una versión 1.1.
- **Reconocimiento de logos comerciales en el ticket:** inviable con frameworks nativos sin entrenar un modelo CoreML específico. El esfuerzo de etiquetado y entrenamiento supera el valor entregable en el alcance actual. Lo suplimos con el diccionario de palabras clave en el texto extraído por OCR.

3. Alcance y MVP

El MVP que definimos para GastoClaro AI cubre el ciclo completo de registro, análisis y comprensión del gasto personal con énfasis en gastos hormiga. Su objetivo es validar la hipótesis de valor con usuarios reales antes de invertir en integraciones bancarias o modelos predictivos.

3.1 Alcance incluido en el MVP

- Registro manual de gastos con todos los campos descritos en RF-01.
- Escaneo de tickets con OCR y sugerencia de categoría.
- Persistencia local de todos los datos con SwiftData.
- Dashboard, lista de gastos, vista de detalle y gestión de categorías.
- Insights automáticos calculados localmente.
- Comparación mensual con navegación por año.
- Resumen narrativo con OpenAI configurable por el usuario.
- Configuración de presupuesto mensual y métricas resumen.

3.2 Alcance excluido del MVP

- Conexión bancaria automatizada.
- Reconocimiento de voz.
- Modelos de predicción propios.
- Sincronización en la nube entre dispositivos.
- Versión Android.
- Widgets de iOS y app para Apple Watch.

3.3 Criterios de aceptación del MVP

Consideramos que el MVP cumple su propósito cuando se validen los siguientes criterios con usuarios beta:

- Más del 70% de los tickets escaneados son procesados correctamente sin intervención manual relevante.
- El tiempo promedio para registrar un gasto manual es menor a 15 segundos.
- Más del 60% de los usuarios beta abren la app al menos 3 veces por semana en el primer mes.
- Al menos el 30% de usuarios beta activa la generación de resúmenes con IA dentro de las primeras 2 semanas.
- Cero pérdida de datos por fallas de la app en pruebas de carga con 5,000 gastos por usuario.

4. Wireframes de la app

A continuación presentamos las 10 pantallas que cubren el flujo principal y los flujos secundarios relevantes de GastoClaro AI. Nueve de ellas son capturas reales de la aplicación funcionando, tal como las presentamos en el Hack, con datos de muestra cargados. La última corresponde al diseño de la pantalla de configuración de la API key. Cada pantalla está disponible también en alta resolución dentro de la carpeta de anexos, nombradas wf01_dashboard hasta wf10_api_key; las capturas reales en formato JPG y la pantalla de diseño en PNG y SVG.



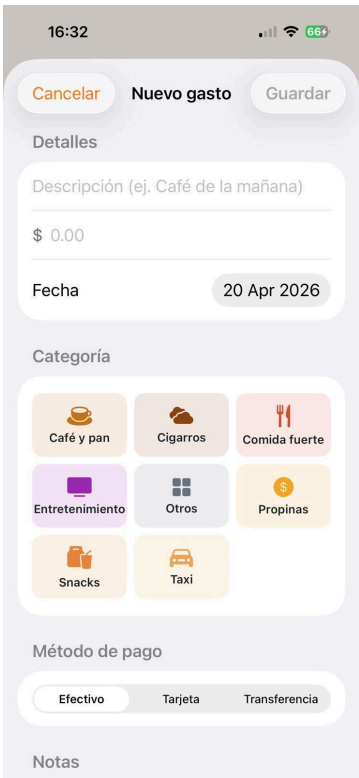
W01. Dashboard

Figura 2. Resumen del mes, gastos hormiga, presupuesto y accesos rápidos.



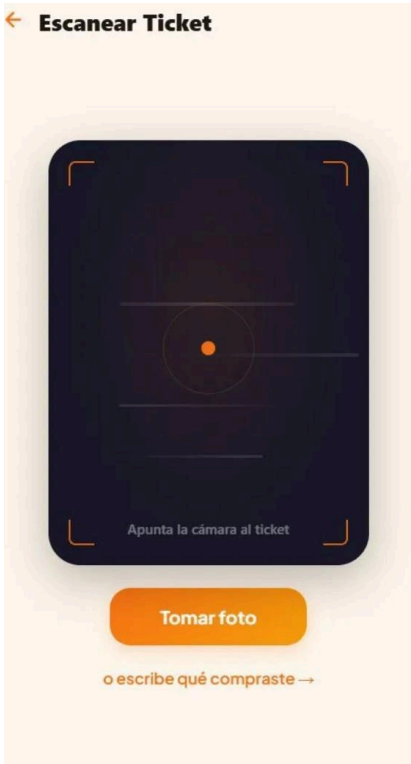
W02. Lista de gastos

Figura 3. Histórico agrupado por día, con buscador y filtro por categoría.



W03. Nuevo gasto

Figura 4. Alta manual con todos los campos y selector visual de categoría.



W04. Escanear ticket

Figura 5. Cámara con marco de captura para alinear el ticket.



W05. Insights

Figura 6. Acceso al resumen con IA, comparación mensual y distribución.



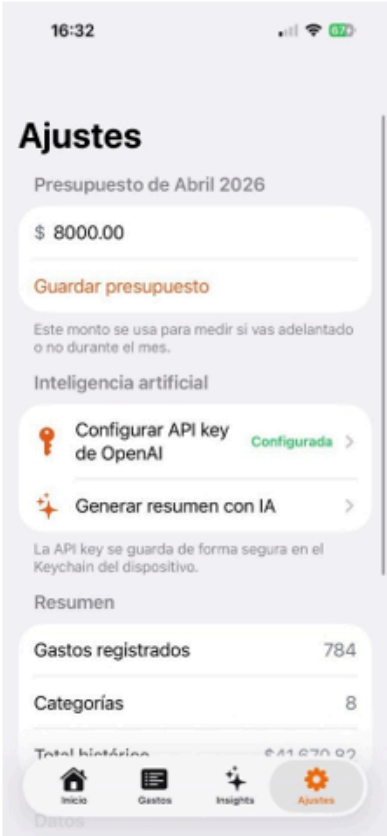
W06. Resumen con IA

Figura 7. Resumen del mes con aviso de privacidad de datos enviados.



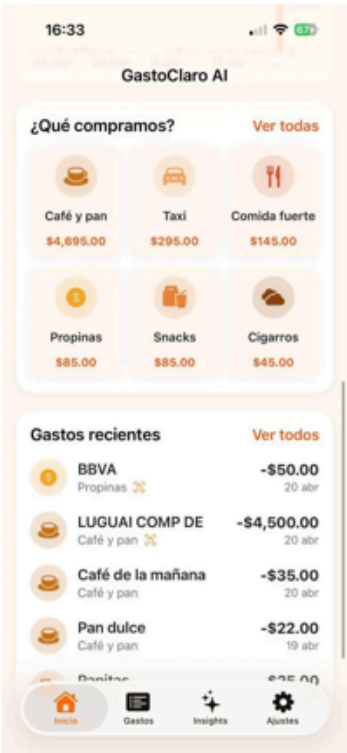
W07. Comparación mensual

Figura 8. Gráfica de barras de 12 meses con navegación por año.



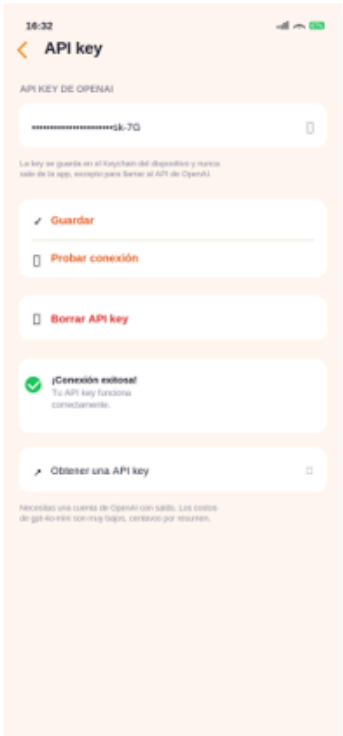
W08. Ajustes

Figura 9. Presupuesto, integración con IA y estadísticas de uso.



W09. Categorías y recientes

Figura 10. Desglose por categoría y lista de gastos recientes



W10. Configurar API key

Figura 11. Diseño para ingresar, validar y eliminar la API key de OpenAI

5. Flujo de navegación

La navegación de la app se organiza alrededor de una barra inferior con cuatro pestañas: Inicio, Gastos, Insights y Ajustes. Desde cada una se accede a las pantallas modales y de detalle.

5.1 Flujos principales

Identificamos cuatro flujos principales que representan el 90% del uso esperado:

- **Flujo A. Registro manual:** Dashboard W01 → toca Registrar → Nuevo gasto W03 → llena campos → toca Guardar → regresa a W01 con el gasto añadido.
- **Flujo B. Registro por ticket:** Dashboard W01 → toca Escanear ticket → Cámara W04 → toca Tomar foto → OCR procesa → muestra los datos detectados para confirmar → toca Guardar gasto → regresa a W01.
- **Flujo C. Consulta y análisis:** Pestaña Insights W06 → toca Resumen con IA → AISummaryView W07 → toca Generar resumen → modelo procesa → muestra resumen narrativo.
- **Flujo D. Comparación histórica:** Pestaña Insights W06 → toca Comparación mensual → Comparación W08 → navega entre años con chevrons → toca una barra para ver desglose por categoría.

5.2 Gestos especiales

La app aprovecha gestos nativos de iOS para algunas interacciones:

- **Swipe horizontal sobre fila de gasto:** revela acciones de eliminar con confirmación. Es el gesto estándar de iOS y los usuarios lo descubren intuitivamente.
- **Pull to refresh en la lista de gastos:** recarga los insights automáticos. Útil después de un período sin abrir la app.
- **Long press sobre una barra de la gráfica mensual:** muestra el valor exacto del mes en un tooltip flotante.
- **Tap sobre una barra mensual:** selecciona el mes y muestra su desglose por categoría en la sección inferior.
- **Tap sobre el icono del ojo en el campo de API key:** alterna entre mostrar y ocultar el texto del campo.
- **Swipe down sobre cualquier modal:** cierra el modal y regresa a la vista previa, gesto estándar de los sheets de iOS.

6. Documentación por pantalla

Para cada pantalla documentamos cuatro aspectos: funcionalidad implementada, información y tipos de datos manejados, servicios y operaciones de datos, y uso de almacenamiento local.

6.1 W01. Dashboard

Funcionalidad: vista de resumen del mes en curso. Muestra total, presupuesto, gastos hormiga, proyección anual y top categorías. Es la primera pantalla al abrir la app.

Dato	Tipo	Vigencia	Origen	Operación
Total del mes	Double	Tiempo real	Cálculo local	Consulta
Presupuesto	Double	Persistente mensual	SwiftData local	Consulta
Gastos hormiga del mes	Double	Tiempo real	Cálculo local	Consulta
Proyección anual	Double	Tiempo real	Cálculo local	Consulta

Dato	Tipo	Vigencia	Origen	Operación
Lista de categorías top	Array<Category>	Tiempo real	SwiftData local	Consulta

Almacenamiento local: todos los datos provienen de la base SwiftData del dispositivo. No hay llamadas a servicios externos en esta pantalla. Los cálculos se hacen on-the-fly al cargar la vista.

6.2 W02. Lista de gastos

Funcionalidad: histórico completo de gastos con búsqueda y filtro. Permite navegar al detalle de cada gasto.

Dato	Tipo	Vigencia	Origen	Operación
Lista de gastos	Array<Expense>	Persistente	SwiftData local	Consulta
Texto de búsqueda	String	Sesión	Estado de vista	Consulta/filtro
Categoría filtrada	Category?	Sesión	Estado de vista	Consulta/filtro
Subtotal por día	Double	Tiempo real	Cálculo local	Consulta

Almacenamiento local: SwiftData mantiene todos los gastos. Las consultas se construyen con FetchDescriptor de SwiftUI Query, que es reactivo a cambios.

6.3 W03. Nuevo gasto

Funcionalidad: formulario para registrar un gasto manualmente. Valida que descripción, monto y categoría sean obligatorios.

Dato	Tipo	Vigencia	Origen	Operación
Descripción	String	Sesión hasta guardar	Input del usuario	Registro
Monto	Double	Sesión hasta guardar	Input del usuario	Registro
Fecha	Date	Sesión hasta guardar	DatePicker	Registro
Categoría	Category	Sesión hasta guardar	Selección visual	Registro
Método de pago	String	Sesión hasta guardar	Picker	Registro
Notas	String	Sesión hasta guardar	TextField	Registro

Almacenamiento local: al guardar, se inserta un objeto Expense en SwiftData. La pantalla se cierra y regresa al dashboard que se actualiza reactivamente.

6.4 W04. Escaneo de ticket

Funcionalidad: captura imagen del ticket con cámara, ejecuta OCR con framework Vision, extrae comercio, monto y categoría con heurísticas, y prellena el formulario de nuevo gasto.

Dato	Tipo	Vigencia	Origen	Operación
Imagen del ticket	UIImage	Procesamiento	Cámara on-device	Procesar
Texto OCR	Array<String>	Procesamiento	Vision framework	Procesar
Comercio detectado	String	Sesión	Heurística local	Sugerencia
Monto detectado	Double	Sesión	Regex sobre texto OCR	Sugerencia
Categoría sugerida	String	Sesión	Diccionario de keywords	Sugerencia

Almacenamiento local: la imagen del ticket NO se persiste; se procesa en memoria y se descarta. Solo el gasto resultante se guarda en SwiftData.

Sensores y permisos: usa la cámara trasera del iPhone con permiso NSCameraUsageDescription. Si el usuario está en simulador o sin cámara, ofrece PhotosPicker para seleccionar imagen del rollo, lo que requiere NSPhotoLibraryUsageDescription.

6.5 W06. Insights

Funcionalidad: muestra insights generados localmente con InsightsEngine, accesos al resumen con IA y a la comparación mensual, y gráfica de distribución por categoría.

Dato	Tipo	Vigencia	Origen	Operación
Insights generados	Array<Insight>	Tiempo real	InsightsEngine local	Consulta
Distribución por categoría	Array<(Cat, Double)>	Tiempo real	Cálculo local	Consulta
Estado de API key	Bool	Persistente	Keychain	Consulta

Almacenamiento local: SwiftData para los datos del cálculo, Keychain solo se consulta para saber si mostrar la opción de IA con estado configurado.

6.6 W07. Resumen con IA

Funcionalidad: genera un resumen narrativo del mes usando el API de OpenAI. Construye un payload con datos agregados, llama al endpoint y muestra el resultado en Markdown.

Dato	Tipo	Vigencia	Origen	Operación
Snapshot financiero	FinancialSnapshot	Sesión	Cálculo local	Consulta
API key	String	Persistente	Keychain	Consulta
Prompt enviado	String	Sesión	Construido localmente	Envío
Respuesta del modelo	String	Sesión	OpenAI API	Consulta remota

Servicio externo: POST a <https://api.openai.com/v1/chat/completions>. Modelo gpt-4o-mini. Autenticación con header Authorization: Bearer <api_key>. Formato JSON. Costo aproximado: 0.15 USD por millón de tokens de entrada, 0.60 USD por millón de tokens de salida. Cada

resumen consume aproximadamente 500 tokens de entrada y 300 de salida, lo que equivale a menos de 0.001 USD por resumen.

Almacenamiento local: el resumen generado se mantiene en memoria solo durante la sesión; no se persiste. Si el usuario sale y entra, debe regenerar.

6.7 W08. Comparación mensual

Funcionalidad: gráfica de barras con los 12 meses del año seleccionado. Permite navegar entre años y consultar el desglose por categoría del mes seleccionado.

Dato	Tipo	Vigencia	Origen	Operación
Año seleccionado	Int	Sesión	Estado de vista	Consulta/filtro
Totales mensuales	Array<MonthTotal>	Tiempo real	Cálculo local	Consulta
Mes seleccionado	Int	Sesión	Tap del usuario	Consulta/filtro
Desglose por categoría	Array<(Cat, Double)>	Tiempo real	Cálculo local	Consulta

Almacenamiento local: todos los datos provienen de SwiftData con consultas agregadas por año y mes.

6.8 W09 y W10. Ajustes y API key

Funcionalidad: gestión del presupuesto mensual, configuración de la API key de OpenAI y opciones de borrado.

Dato	Tipo	Vigencia	Origen	Operación
Presupuesto	Double	Persistente mensual	SwiftData	CRUD
API key	String	Persistente segura	Keychain	CRUD
Estadísticas resumen	Triple<Int,Int,Double>	Tiempo real	Cálculo local	Consulta

Almacenamiento local: el presupuesto se guarda como objeto MonthlyBudget en SwiftData. La API key NO se guarda en UserDefaults ni en SwiftData; va al Keychain con la flag kSecAttrAccessibleAfterFirstUnlock para que solo sea accesible después del primer desbloqueo del dispositivo.

7. Flujo de datos y cambios de estado

La arquitectura de datos de GastoClaro AI es local-first: todos los datos del usuario viven en el dispositivo y se sincronizan con la UI de forma reactiva.

7.1 Capas de datos

- **Capa de modelo:** definida por las clases @Model Expense, Category y MonthlyBudget. SwiftData se encarga de la persistencia.
- **Capa de servicios:** InsightsEngine para cálculos, TicketScannerService para OCR, AIService para llamadas a OpenAI, APIKeyStore para Keychain.
- **Capa de vistas:** SwiftUI Views consumen los modelos con @Query y reaccionan automáticamente a cambios.

7.2 Cambios de estado relevantes

- **Estado inicial:** al abrir la app por primera vez, SeedData siembra 8 categorías y un presupuesto inicial. Esto evita pantallas vacías que confunden al usuario.
- **Estado con datos:** cada inserción, edición o borrado de Expense dispara la actualización reactiva de todas las vistas que dependen del modelo.
- **Estado de carga del OCR:** máquina de estados idle → processing → success/failure. La vista muestra spinner, resultado o error según el caso.
- **Estado de carga de IA:** máquina de estados similar: noKey → empty → loading → summary/error. Cada estado tiene una vista específica.
- **Estado de autenticación de API key:** binario configurada/sin configurar. Se consulta al Keychain con APIKeyStore.hasKey.

8. Dispositivos, orientación y sensores

8.1 Dispositivos soportados

- **iPhone:** modelo mínimo iPhone 11. Pantalla mínima 5.4 pulgadas, pantalla óptima 6.1 a 6.7 pulgadas. Razón: a partir de iPhone 11 todos los modelos tienen Neural Engine y soportan iOS 17.
- **iPad:** compatibilidad como app universal en fase 2. La pantalla más grande permitirá mostrar dashboard y lista en paralelo con NavigationSplitView.
- **Apple Watch:** previsto en fase 2 para registro rápido por voz y vista de total del mes.

8.2 Orientación de pantalla

La app está pensada principalmente para orientación vertical (portrait), que es la forma natural de uso de finanzas personales en iPhone. Habilitamos orientación horizontal (landscape) solamente en las pantallas de gráficas para aprovechar el espacio cuando el usuario quiere ver más detalle, y en la cámara del escáner que se beneficia de la rotación natural del dispositivo durante la captura.

8.3 Sensores utilizados

- **Cámara trasera:** para captura de tickets en W04. Se accede vía VNDocumentCameraViewController de VisionKit, que provee detección automática de bordes, autoenfoco y corrección de perspectiva sin que tengamos que implementarlo manualmente. Requiere permiso NSCameraUsageDescription.
- **Neural Engine:** se aprovecha indirectamente al ejecutar VNRecognizeTextRequest del framework Vision. El motor neural acelera la inferencia del OCR sin necesidad de configuración adicional.
- **Photo Library:** como alternativa a la cámara cuando no está disponible, principalmente en simulador. Requiere permiso NSPhotoLibraryUsageDescription.
- **Acelerómetro:** no se usa directamente, pero UIKit lo usa internamente para soportar el cambio de orientación de las pantallas que sí permiten landscape.

9. Stack técnico y herramientas

9.1 Lenguajes de programación

Toda la app está escrita en Swift 5.9 o superior. No usamos Objective-C ni librerías que lo requieran. La elección se debe a tres razones: integración nativa con SwiftUI, seguridad de tipos en tiempo de compilación, y soporte de async/await que simplifica las llamadas asíncronas como las del API de OpenAI.

9.2 Frontend

- **SwiftUI:** framework declarativo para la interfaz. Permite construir vistas reactivas con menos código que UIKit.
- **Swift Charts:** para gráfica circular de distribución, gráfica de barras de tendencia y gráfica de comparación mensual. Disponible desde iOS 16.
- **VisionKit:** para el escáner de documentos con detección automática de bordes.
- **Vision:** para el reconocimiento óptico de caracteres.
- **PhotosUI:** para PhotosPicker como alternativa a la cámara.

9.3 Backend y persistencia

- **SwiftData:** ORM declarativo para persistencia local con modelos anotados con @Model. Reemplaza a Core Data con un API más moderno.
- **Keychain Services:** para almacenamiento seguro de la API key con la flag kSecAttrAccessibleAfterFirstUnlock.
- **URLSession con async/await:** para las llamadas HTTP al API de OpenAI.
- **Foundation:** para NumberFormatter, DateFormatter, Calendar y manejo de regex en el parsing de montos.

9.4 Servicios externos

- **OpenAI Chat Completions API:** <https://api.openai.com/v1/chat/completions> con el modelo gpt-4o-mini. Autenticación Bearer. Formato JSON. Costo: 0.15 USD por millón de tokens de entrada y 0.60 USD por millón de tokens de salida.

9.5 Herramientas de desarrollo

- **Xcode 15 o superior:** IDE oficial de Apple. Incluye simulador de iPhone, debugger, profiler con Instruments y soporte para previews en SwiftUI.
- **Swift Package Manager:** gestor de dependencias nativo. Aunque la app actual no usa paquetes externos, queda preparado para integrar Sentry, Firebase Crashlytics o similares en fases futuras.
- **Git con GitHub:** control de versiones. Usamos pull requests y revisión cruzada entre los tres miembros del equipo.
- **TestFlight:** para distribución beta antes de publicación en App Store.

10. Arquitectura y metodologías

10.1 Arquitectura general

Adoptamos una arquitectura MVVM ligera, adaptada al modelo declarativo de SwiftUI. La separación es la siguiente:

- **Modelos:** definidos en /Models. Solo contienen propiedades y validaciones de dominio. No saben de SwiftUI.
- **Vistas:** definidas en /Views. Son struct: View de SwiftUI. Consumen modelos a través de @Query y propagan acciones por callbacks o environments.
- **Servicios:** definidos en /Services. Encapsulan lógica que va más allá de la persistencia: OCR, IA, Keychain, motor de insights.
- **Utilidades:** definidas en /Utilities. Theme, Formatters y datos semilla.

10.2 Patrones de diseño aplicados

- **Patrón Repository:** implícito en SwiftData a través de FetchDescriptor. Aísla la lógica de consulta de los detalles de almacenamiento.
- **Patrón Service:** los servicios son enum sin estado con métodos estáticos. Esto los hace fácilmente testeables y evita acoplamiento por inyección compleja.

- **Patrón State Machine:** en TicketScannerView y AISummaryView usamos enum ScanState y AIState con cuatro casos: idle, processing, success, failure. Cada caso renderiza una vista distinta.
- **Patrón Observer:** implícito en @Query y @Observable de SwiftUI. Las vistas se actualizan automáticamente cuando el modelo cambia.
- **Patrón Factory para insights:** InsightsEngine genera dinámicamente la lista de insights según las condiciones detectadas, sin que la vista conozca la lógica de generación.

10.3 Metodología de desarrollo

- **Scrum ligero:** adaptamos Scrum al formato del Hack con sprints de medio día. Tres reuniones rápidas al día: planificación matutina, sincronización a media tarde, revisión nocturna.
- **Programación en parejas:** para las partes más críticas como el OCR y la integración con OpenAI, trabajamos en pareja para asegurar calidad y compartir conocimiento.
- **Trunk based development:** una sola rama principal, commits pequeños y frecuentes, revisión asincrónica vía pull request.

11. Administración del proyecto

La gestión del proyecto en el Hack fue tan importante como el código. La organizamos así:

11.1 Herramientas

- **Notion:** como espacio único de planificación. Backlog priorizado, documentación viva, decisiones tomadas y enlaces a recursos.
- **GitHub Projects:** tablero Kanban con cuatro columnas: backlog, en progreso, revisión, hecho. Cada tarjeta es un issue de GitHub vinculado a su pull request.
- **Slack:** para comunicación asincrónica y notificaciones automáticas de GitHub.
- **Google Meet:** para los tres standups diarios.

11.2 Estructura del backlog

Organizamos el backlog en cinco épicas, cada una con sus historias de usuario:

- Épica 1: Modelo de datos y persistencia local.
- Épica 2: Registro y consulta de gastos.
- Épica 3: Escaneo de tickets con OCR.
- Épica 4: Insights y comparación mensual.
- Épica 5: Integración con IA generativa.

11.3 Roles dentro del Hack

Aunque todos contribuimos en todo, asignamos responsabilidades primarias para evitar duplicaciones:

- **Héctor Guerrero:** arquitectura general, modelos SwiftData, servicios y motor de insights.
- **Yuichi Miyasaki:** interfaz de usuario, animaciones, integración con cámara y Vision, diseño visual.
- **Cinthia Sampayo:** integración con OpenAI, manejo de Keychain, documentación del producto y presentación final.

12. Uso de IA en el desarrollo

La IA fue parte activa de nuestro proceso de desarrollo, no solo del producto final. La aprovechamos en tres frentes:

12.1 Como asistente de programación

Usamos asistentes de IA tipo Claude y Copilot para generar boilerplate, sugerir refactorizaciones y explicar APIs nativas que no conocíamos a fondo. En particular, la documentación generativa nos ayudó con VisionKit, SwiftData y los detalles del API de Chat Completions de OpenAI.

12.2 Como herramienta de diseño y diagramación

Generamos prototipos y wireframes con asistencia de IA, lo que nos permitió iterar el diseño visual rápidamente. También generamos datos de muestra sintéticos que reproducen patrones reales de gasto mexicano para que el dashboard se viera convincente desde el primer arranque.

12.3 Como motor del producto final

La aplicación incorpora GPT-4o mini como motor del resumen financiero personalizado. Esto nos permitió ofrecer una capacidad de análisis narrativo que sería imposible construir con reglas explícitas. La IA dentro del producto está aislada en un servicio AIService, con un prompt estructurado que limita la entrada a datos agregados.

13. Permisos y consideraciones para la publicación

13.1 Permisos del sistema

La app declara los siguientes permisos en su Info.plist:

- **NSCameraUsageDescription:** "Se usa para escanear tus tickets y registrar gastos automáticamente." Necesario para abrir VNDocumentCameraViewController.
- **NSPhotoLibraryUsageDescription:** "Se usa para seleccionar fotos de tickets desde tu galería cuando la cámara no está disponible." Necesario para PhotosPicker.

13.2 Privacy Nutrition Labels

App Store exige declarar qué datos recolectamos y para qué. Nuestra declaración es:

- Datos recolectados que se asocian al usuario: ninguno. Toda la información financiera vive solo en el dispositivo del usuario.
- Datos recolectados que NO se asocian al usuario: ninguno. No hay analytics de terceros en la versión inicial.
- Datos NO recolectados: identidad, contactos, ubicación, salud, finanzas, identificadores, datos de uso, diagnósticos, otros. Estos son los datos que App Store espera ver declarados como no recolectados.

13.3 Cumplimiento normativo

- **Ley Federal de Protección de Datos Personales en Posesión de Particulares en México:** no aplica con rigor porque la app no almacena datos personales en servidores propios. Sin embargo, publicamos un aviso de privacidad simple dentro de la app que explica el modelo on-device.
- **Reglas de App Store sobre uso de cámara y datos financieros:** cumplimos al declarar el permiso de cámara con un mensaje claro, no transmitir datos financieros del usuario a terceros sin su consentimiento explícito, y no usar las credenciales de la API key del usuario para fines distintos a los declarados.
- **Política sobre IA generativa de Apple:** App Store permite apps que usan modelos generativos externos siempre que se filtren contenidos sensibles y haya supervisión humana. Nuestro uso es para análisis financiero, no para generación de contenido público, así que está dentro de los términos aceptados.

13.4 Requisitos previos a la publicación

- Cuenta Apple Developer Program con costo anual de 99 USD.
- Certificado de distribución y perfil de aprovisionamiento.
- Capturas de pantalla en al menos tres tamaños de iPhone.

- Texto promocional, descripción, palabras clave y URL de soporte.
- Validación con TestFlight y al menos 25 usuarios beta durante 2 semanas.
- Revisión del proceso de App Store, que típicamente tarda entre 24 horas y 7 días.

14. Equipo de trabajo y roles

Para llevar el MVP a producción real fuera del contexto del Hack, identificamos los roles necesarios en un equipo profesional:

Rol	Responsabilidad principal	Dedicación
Product Manager	Define el roadmap, prioriza el backlog, conduce entrevistas con usuarios.	Tiempo completo
Tech Lead iOS	Arquitectura, calidad del código, decisiones técnicas, mentoría.	Tiempo completo
Desarrollador iOS Senior	Implementación de vistas y servicios, integración con OCR y IA.	Tiempo completo
Desarrollador iOS Junior	Implementación de pantallas y pruebas unitarias.	Tiempo completo
Diseñador UX/UI	Diseño visual, prototipos, sistema de diseño, pruebas de usabilidad.	Medio tiempo
QA Tester	Plan de pruebas, automatización con XCTest y XCUITest, regresión.	Medio tiempo
DevOps / SRE	Pipeline de CI/CD, gestión de TestFlight y App Store Connect, monitoreo.	Cuarto de tiempo
Especialista en IA / Datos	Mejora del prompt, evaluación de calidad de resúmenes, exploración de modelos.	Cuarto de tiempo

15. Estimaciones de tiempo y costos

15.1 Tiempo de desarrollo

Estimación para llevar el MVP del estado actual al lanzamiento comercial:

Fase	Duración	Entregable
Fase 0. Estado actual (Hack)	Completado	MVP funcional con datos de muestra.
Fase 1. Pulido y QA	4 semanas	Cobertura de tests, manejo de errores, accesibilidad completa.
Fase 2. Beta con TestFlight	6 semanas	25 usuarios beta, ajustes según feedback, métricas iniciales.
Fase 3. Localización y App Store	3 semanas	Captura de assets, descripciones, revisión y publicación.
Fase 4. Iteración postlanzamiento	12 semanas	Mejoras según uso real, conexión bancaria piloto, widgets.

Suma total: aproximadamente 25 semanas desde el final del Hack hasta una versión 1.0 con tracción inicial.

15.2 Costos de desarrollo

Estimación con salarios promedio de mercado mexicano en 2026 para un equipo profesional:

Concepto	Mensual MXN	Anual MXN
Tech Lead iOS	\$70,000	\$840,000
Desarrollador Senior iOS	\$50,000	\$600,000
Desarrollador Junior iOS	\$25,000	\$300,000
Diseñador UX/UI (medio tiempo)	\$18,000	\$216,000
QA Tester (medio tiempo)	\$15,000	\$180,000
Product Manager	\$45,000	\$540,000
DevOps (cuarto de tiempo)	\$12,000	\$144,000
Apple Developer Program	—	\$2,000
Infraestructura mínima	\$3,000	\$36,000
TOTAL	\$238,000	\$2,858,000

15.3 Costos de operación e implementación

- **OpenAI gpt-4o-mini:** estimando 10,000 usuarios activos mensuales que generan 5 resúmenes al mes en promedio, el costo es de aproximadamente 40 a 60 USD mensuales, prácticamente despreciable.
- **Apple Developer Program:** 99 USD anuales, aproximadamente 2,000 pesos mexicanos.
- **Infraestructura adicional (fase 2):** para sincronización CloudKit, monitoreo Sentry y backend de telemetría agregada, estimamos 200 a 500 USD mensuales en función del crecimiento.

15.4 Costos de mantenimiento

A partir del primer año de operación, el costo de mantenimiento se reduce significativamente porque ya no se necesita el equipo completo en modo construcción. Estimamos un costo mensual de mantenimiento de aproximadamente 80,000 pesos mexicanos, equivalente a 960,000 anuales. Incluye un desarrollador iOS de tiempo completo, soporte parcial de QA y de producto, y los costos de operación de OpenAI e infraestructura.

16. Referencias

Las fuentes consultadas durante la investigación, desarrollo y documentación son:

- BBVA México, 2023. ¿Cómo entender y controlar los gastos hormiga en México? bbva.com/es/mx/salud-financiera
- CONDUSEF. Revista Proteja su Dinero, sección Gasto hormiga. revista.condusef.gob.mx
- El Financiero, julio 2025. Gastos hormiga te cuestan de 900 a mil 500 pesos al mes. elfinanciero.com.mx
- INEGI. Encuesta Nacional de Ingresos y Gastos de los Hogares ENIGH 2024.
- Pluxee México, diciembre 2025. Qué son los gastos hormiga y cómo eliminarlos en 2026.
- Encuesta Nacional de Inclusión Financiera ENIF 2021, INEGI.

- Apple Inc. Documentación oficial de SwiftUI, SwiftData, Vision, VisionKit y Swift Charts. developer.apple.com
- Apple Inc. App Store Review Guidelines. developer.apple.com/app-store/review/guidelines
- Apple Inc. Human Interface Guidelines. developer.apple.com/design/human-interface-guidelines
- OpenAI, 2026. Documentación oficial del modelo gpt-4o-mini. platform.openai.com/docs/models/gpt-4o-mini
- OpenAI Pricing. platform.openai.com/docs/pricing
- Salarios IT en México 2026, encuesta de Glassdoor y Hays.
- Comparativa de apps de finanzas personales: Fintonic, Mobills, Wallet, Monefy y Splitwise.

17. Anexos

Los siguientes archivos complementan este trabajo y se entregan junto al documento:

- **Anexo A. Pantallas en alta resolución:** 10 archivos individuales en la carpeta wireframes/. Nueve son capturas reales de la app en formato JPG, nombradas wf01_dashboard.jpg hasta wf09_dashboard_categorias.jpg, y la pantalla de diseño de API key en wf10_api_key.png y wf10_api_key.svg.